Singleton Design Pattern

1. Classification, Intent & Exam Triggers

- **Category:** Creational Design Pattern (controls how objects are instantiated in memory — restricting instantiation to a single shared object or a fixed number of objects N).
- **Intent (GoF Definition):** Ensures a class has one and only one instance throughout the application lifecycle, and provides a global point of access to that instance.
- **Primary Problem Solved (Multiple Object Chaos):** Prevents multiple departments/modules from calling 'new' and creating separate manager objects for a single shared resource, which causes: (1) Resource & Transaction Conflicts (printer freezes, simultaneous file/DB locks, duplicate job execution), (2) Data & Status Inconsistency (each object holds its own queue/state, showing conflicting statuses across departments), (3) Memory & Performance Waste (redundant heavy objects in heap memory), and (4) Difficult System Maintenance.
- **Core OOP Principles:** Applies Encapsulation ('private' constructor blocks external 'new' calls and hides the instance variable), Class-Level State ('static' variable and 'static getInstance()' belong to the class itself rather than individual objects, ensuring only 1 copy exists), and Final/Sealed Protection ('final' class in Java / 'sealed' in C# prevents nested subclasses from bypassing the private constructor).
- **How to Spot in Exam Questions:**
- (1) Single-Instance Trigger: 'Ensure all logs are written to the same file', 'A single global tax calculator across all checkout counters', 'A single global OrderID generator', 'Salary calculation run by only one user at a time', 'Multiple departments send print jobs to a single high-speed printer'.
- (2) Extended Singleton / Multiton Trigger: 'Allow a fixed number N of instances instead of 1' — e.g., 'Fantasy cricket team with max 5 batsmen, 5 bowlers, 1 wicketkeeper, and 1 captain' or N=3/4/6/10/20 printers, gates, ATMs, or regional servers.

2. Pattern Structure, Participants & UML Relationships

Member / Participant Role	Modifiers & UML Notation	Responsibility & Exam Significance
1. Singleton Class (e.g., PrinterManager, Logger, TaxCalculator, OrderIdGenerator)	public final class (sealed in C#)	Manages its own sole instance. Declared 'final' ('sealed') so inner/nested subclasses cannot inherit from it and create extra instances.
2. Static Instance Field (Self-Reference)	- instance: Singleton (private static [volatile] — underlined in UML)	Holds the single shared reference belonging to the class itself. Because it is 'static', only one copy exists in memory across the entire application.
3. Private Constructor	- Singleton()	Prevents external classes from instantiating objects using 'new Singleton()'. Guarantees

	(private — minus '-' sign in UML)	the class itself controls when and how the object is created.
4. Global Access Method	+ getInstance(): Singleton (public static — underlined in UML)	Checks 'if (instance == null)' — creates the instance on the first call (lazy initialization) and returns the exact same existing object on all subsequent calls.
5. Multiton (Extended Singleton for N Objects)	- instances: List<T> - MAX_INSTANCES: int - nextIndex: int + getInstance(): T	Replaces the single 'instance' field with a static List or Map capped at 'MAX_INSTANCES = N'. Distributes the N instances via Round-Robin ('nextIndex = (nextIndex + 1) % MAX') or key lookup.

High-Yield Lecture & Textbook Q&A Rules

- **Why is the constructor private? What if it were public? :** A private constructor stops clients from calling 'new Singleton()'. If it were public, calling 'new Singleton()' twice would allocate two separate objects in memory, causing 's1 == s2' to print 'false' instead of 'true'.
- **Why must 'instance' and 'getInstance()' be static?:** Because the constructor is private, no object exists yet for a client to call a normal method on. Making 'getInstance()' static lets clients call 'Singleton.getInstance()' directly on the class name, and a static method can only access a static 'instance' variable.
- **Why is Lazy Initialization NOT thread-safe, and how do we fix it?:** In a multithreaded environment, two threads can evaluate 'if (instance == null)' at the exact same moment and create two separate instances. Fix it using Double-Checked Locking with a 'private static volatile Singleton instance' field and a 'synchronized' (or C# 'lock') block so locking overhead only happens on the very first creation.
- **Why use the 'volatile' keyword in Double-Checked Locking?:** 'volatile' prevents compiler instruction reordering and CPU caching, ensuring that write assignment to 'instance' completes in main memory before any other thread reads it.
- **Why use 'final' (Java) / 'sealed' (C#) even when the constructor is private?:** While a private constructor stops external classes from subclassing, a nested derived class inside the Singleton class ('public class NestedDerived extends Singleton') CAN access the private constructor and create multiple objects. Marking the class 'final' / 'sealed' blocks this loophole.

3. Implementation Code Structure

```
// Standard Lazy Singleton Implementation
public final class PrinterManager {
  // 1. Private static variable holds the single shared instance in memory
  private static PrinterManager instance;

  // 2. Private constructor prevents external instantiation via 'new PrinterManager()'
  private PrinterManager() {
    System.out.println("Initializing single central PrinterManager queue...");
```

```
  }

  // 3. Public static global access point (Lazy Initialization)
  public static PrinterManager getInstance() {
    if (instance == null) {
      instance = new PrinterManager(); // Created only on the first call
    }
    return instance; // Returns the same existing object on all future calls
  }

  public void submitPrintJob(String dept, String document) {
    System.out.println("[" + dept + "] Queued job: " + document);
  }
}

// Client Execution & Memory Reference Trace (N-02 Slides 13-14)
public class Main {
  public static void main(String[] args) {
    PrinterManager labManager = PrinterManager.getInstance();     // First call: creates object #1
    PrinterManager libraryManager = PrinterManager.getInstance(); // Second call: returns object #1

    labManager.submitPrintJob("Computer Lab", "Thesis.pdf");
    libraryManager.submitPrintJob("Library", "ExamPaper.pdf");

    // Reference equality check: both variables point to the exact same object in memory
    System.out.println(labManager == libraryManager); // Output: true
  }
}
```

Extending Singleton to Multiton

```
// 1. Multiton Pattern with Round-Robin Allocation (N-02 Slide 17 & In-Class Groups 2, 6)
import java.util.*;
public final class CloudPrinterManager {
  private static final int MAX_PRINTERS = 3; // Fixed cap of N instances (e.g., 3, 6, or 10)
  private static final List<CloudPrinterManager> printers = new ArrayList<>();
  private static int nextPrinter = 0;
  private final int id;

  private CloudPrinterManager(int id) { this.id = id; }

  public static synchronized CloudPrinterManager getPrinter() {
    if (printers.size() < MAX_PRINTERS) {
      printers.add(new CloudPrinterManager(printers.size() + 1)); // Lazy create up to N
    }
    CloudPrinterManager assigned = printers.get(nextPrinter);
    nextPrinter = (nextPrinter + 1) % MAX_PRINTERS; // Round-Robin: 1 -> 2 -> 3 -> 1
    return assigned;
  }
```

```
}

// 2. Past Paper 2023 Q4.1 Model Solution: Fantasy Cricket Team (Extended Singleton / Multiton)
// Requirement: Max 5 Batsmen, Max 5 Bowlers, Max 1 Wicketkeeper, and 1 Captain (Singleton)
public final class Captain { // Pure Singleton (Max = 1)
  private static Captain instance;
  private Captain() {}
  public static Captain getInstance() {
    if (instance == null) instance = new Captain();
    return instance;
  }
}
public final class PlayerMultiton { // Role-Based Multiton (Bounded per role)
  private static final Map<String, List<PlayerMultiton>> pool = new HashMap<>();
  private static final Map<String, Integer> LIMITS = Map.of(
    "BATSMAN", 5, "BOWLER", 5, "WICKETKEEPER", 1
  );
  private String role, name;
  private PlayerMultiton(String role, String name) { this.role = role; this.name = name; }

  public static PlayerMultiton addPlayer(String role, String name) {
    pool.putIfAbsent(role, new ArrayList<>());
    List<PlayerMultiton> list = pool.get(role);
    if (list.size() < LIMITS.get(role)) {
      PlayerMultiton p = new PlayerMultiton(role, name);
      list.add(p);
      return p;
    }
    throw new IllegalStateException("Maximum limit reached for role: " + role);
  }
}
```

5. Key Comparisons, Trade-Offs & Real-World Challenges

Instance-Limiting Patterns: Singleton vs. Multiton vs. Object Pool vs. Normal Class

Pattern / Construct	Number of Instances	Creation & Reuse Mechanism	Typical Exam Use Case
1. Singleton Pattern	Exactly 1	Created once (lazily or eagerly) via private constructor; shared globally by all callers.	Single DB Connection Manager, 1 High-Speed Printer Queue, Logger, TaxCalculator, OrderIdGenerator, School Principal.
2. Multiton Pattern (Extended Singleton)	Fixed N	Maintains a static List or Map<Key, T> capped at MAX = N; returns instances by key or Round-Robin.	4 Regional Game Servers, 4 City Traffic Zones, 4 Library Counters, 6 Airport Gates, 10 Cloud Printers, 20 ATMs, Cricket Roles.

Lazy Initialization vs. Static/Eager Initialization vs. Double-Checked Locking

Implementation Style	How It Works	Pros & Cons (Exam Trade-Offs)
1. Lazy Initialization (if instance == null)	Instantiates the object inside getInstance() only when requested for the first time.	Pro: Saves memory if the Singleton is never used. Con: NOT thread-safe in multithreaded systems (race condition can create 2 objects).
2. Static / Eager Initialization (private static final instance = new ...)	Instantiates the object immediately when the class is loaded into memory by the JVM / CLR.	Pro: Inherently thread-safe without locking overhead. Con: Less control over instantiation timing — referencing ANY static field (like Singleton.MyInt = 25) triggers instance creation prematurely
3. Double-Checked Locking (volatile + synchronized/lock)	Checks 'if (instance == null)' twice — once before locking and once inside the synchronized block.	Pro: Thread-safe AND lazy; avoids locking overhead on subsequent calls. Con: More verbose code; requires 'volatile' keyword.

Advantages, Disadvantages & Usage Rules

- **Key Advantages:**

  (1) Saves Memory: Only a single instance is allocated in heap memory and reused across the entire application.

  (2) Centralized Control: Provides a single authoritative point of control over shared

hardware, files, configurations, or ID counters.

(3) Global Easy Access: Accessible anywhere via ClassName.getInstance() without passing object references through every method parameter.

(4) Prevents Duplicate Objects & Conflicts: Guarantees data consistency and prevents race conditions or duplicate print/payment jobs.

- **When to Use:** When exactly one instance is needed to coordinate actions, manage a shared resource (printer, DB, file logger), or maintain consistent global state/counters.
- **When NOT to Use:** When multiple independent objects are required, when you have N physical resources (use Multiton instead of 1 bottlenecked Singleton!), or when heavy unit-testing with mock objects is required without interfaces.

Real-World Challenges & How to Overcome Them

Challenge Aspect	How to Explain & Overcome in the Exam
1. Multithreading Race Conditions	Challenge: Two concurrent threads calling getInstance() when instance == null can both pass the check and create two objects. Mechanism to Overcome: Use Double-Checked Locking ('synchronized' block + 'volatile' instance) or Static/Eager initialization.
2. Difficult to Unit Test (Tight Coupling)	Challenge: Because clients call Singleton.getInstance() directly inside their methods, unit tests cannot easily substitute a mock object, and state changes persist across tests. Mechanism to Overcome: Have the Singleton implement an Interface and pass it via Dependency Injection (constructor/parameter) so tests can inject mock implementations.
3. Hidden Dependencies & Global Variable Abuse	Challenge: Global access hides class dependencies inside method bodies rather than declaring them in constructors, acting like a global variable. Mechanism to Overcome: Restrict Singleton strictly to true single-resource managers (Logger, Config, ID Generator) and pass dependencies explicitly where possible.
4. Single-Instance Bottleneck & Single Point of Failure (HA / DR)	Challenge: If a university scales from 1 printer to 4 printers, or needs High Availability (HA) & Disaster Recovery (DR), a single Singleton instance becomes a performance bottleneck or single point of failure. Mechanism to Overcome: Extend Singleton to a Multiton (N instances with Round-Robin load balancing) or an Object Pool with failover replicas.
5. Premature Static Initialization & Nested Subclassing	Challenge: Accessing a dummy static variable (Singleton.MyInt) triggers static initialization prematurely, and an inner subclass can bypass a private constructor. Mechanism to Overcome: Mark the Singleton class 'final' (Java) / 'sealed' (C#) and use Lazy Initialization or a static holder class.